

Uber | LC1462 Course Schedule IV

Graph Visualization

0 → 1 → 2
 ↘ 3

Problem Breakdown

Initially we know:

- 0 prereq of 1
- 1 prereq of 2
- 1 prereq of 3

but we also need:

- 0 prereq of 2
- 0 prereq of 3

State Definition

hence a DP table is needed as parents info will be passed on

DP[node][course] → is course a prereq of node

Kahns is also required, every parent is completely processed before passing info to child

Logic / Transition Rules

hence (P1) // direct prereq

```
dp[child][curr] = true
```

(P2) // inherited preq

```
for (int i=0; i<n; ++i) {  
    if ( dp[curr][i] ) {  
        dp[child][i] = true  
    }  
}
```

C++ Implementation

```
class Solution {  
public:  
    vector<bool> checkIfPrerequisite(int n, vector<vector<int>>&  
prerequisites, vector<vector<int>>& queries) {  
  
        vector<vector<int>> adj(n);  
        vector<int> indeg(n,0);  
        vector<vector<int>> dp(n,vector<int>(n,false));  
        queue<int> q;  
        //dp table for dp[curr][ancestor]  
  
        //initially maan ke chal rhe koi kisi pe depend nhi kr rha  
        for(auto &it:prerequisites){  
            int u=it[0];  
            int v=it[1];  
  
            adj[u].push_back(v);  
            dp[v][u]=1;  
            indeg[v]++;  
        }  
    }  
};
```

```

    for(int i=0;i<n;i++){
        if(indeg[i]==0){
            q.push(i);
        }
    }

    while(!q.empty()){
        int curr=q.front();
        q.pop();

        for(auto &it:adj[curr]){
            dp[it][curr]=1;
            //direct parent

            for(int i=0;i<n;i++){
                if(dp[curr][i]==1){
                    dp[it][i]=1; //agar pawpaw dependent to beta bhi
dependent
                }
            }

            indeg[it]--;
            if(indeg[it]==0){
                q.push(it);
            }
        }
    }

    vector<bool> ans;
    for(auto &it:queries){
        int u=it[0];
        int v=it[1];
        if(dp[v][u]){
            ans.push_back(true);
        }
        else{
            ans.push_back(false);
        }
    }

    return ans;
}
};

```